FIELD SERVICE MANAGEMENT

Complete Backend, Database & API Learning Guide

ASP.NET Core Web API • EF Core • SQL Server • JWT • DTOs • REST APIs • Scalar

Purpose: A beginner-friendly reference for understanding what we built, why we built it, how the database is connected, how DTOs and EF Core work, how controllers call services, how JWT authorization works, and how an API request travels from Scalar/React to SQL Server and back.

Important: The document separates the APIs confirmed from our discussed code/work from the original recommended API catalogue. A recommended endpoint is not automatically claimed as implemented.

1. The Big Picture

Our architecture is a pipeline. Each layer has one main job. The project workflow document describes React → REST API → ASP.NET Core → Controllers → Services → EF Core → SQL Server. ■filecite■turn13file2■L300-L313■

The complete request path

```
React / Scalar
      |
      | HTTP
      v
Controller
      |
      | calls
      v
Service
      |
      | LINQ / EF Core
      v
AppDbContext
      |
      | SQL
      v
SQL Server
      |
      | result
      v
Service -> Controller -> JSON response -> React / Scalar
```

Remember: Controller handles HTTP. Service handles business rules. DbContext handles database communication. Model describes stored data. DTO describes API input/output.

The project development workflow is Model → DbContext → Migration → Database Update → Controller → Scalar Testing → React Integration. ■filecite■turn13file2■L416-L430■

2. Database — Tables, PK, FK and Relationships

Our database design includes Users, CustomerAddresses, Services, Technicians, ServiceRequests, WorkReports, WorkReportParts, Invoices, InvoiceItems, Notifications, StatusHistories, Settings and RefreshTokens. ■filecite■turn13file0■L15-L42■ ■filecite■turn13file0■L54-L84■ ■filecite■turn13file0■L84-L149■ ■filecite■turn13file0■L151-L173■

Parent	Child	FK / relationship	Meaning
Users	Technicians	Technicians.UserId → Users.Id	Technician-specific data belongs to a User.
Users	CustomerAddresses	CustomerId → Users.Id	A customer can have addresses.
Services	ServiceRequests	ServiceId → Services.Id	A request selects a service.
ServiceRequests	WorkReports	ServiceRequestId → ServiceRequests.Id	Work report belongs to a request.
WorkReports	WorkReportParts	WorkReportId → WorkReports.Id	Parts belong to a work report.
ServiceRequests	Invoices	ServiceRequestId → ServiceRequests.Id	Invoice belongs to a request.
Invoices	InvoiceItems	InvoiceId → Invoices.Id	Invoice contains items.
Users	Notifications	UserId → Users.Id	Notification belongs to a user.
ServiceRequests	StatusHistories	ServiceRequestId → ServiceRequests.Id	Request timeline/status changes.

Primary Key (PK): uniquely identifies one row. Example: Users.Id = 10.

Foreign Key (FK): stores the PK value of another table. Example: Technician.UserId = 10 means this Technician belongs to User 10.

Simple relationship

```
Users
```

```
Id    FirstName   Role
10    Rahul       Technician

Technicians
Id    UserId    EmployeeCode
4     10        TECH-004

Technician.UserId points to Users.Id.
```

3. Entity Models and Data Annotations

Entity classes describe the data EF Core maps to SQL Server. Data annotations add validation and schema rules.

Code	Easy meaning
[Key]	Primary key.
[Required]	Value is required / should not be null for applicable types.
[StringLength(20)]	Maximum string length 20.
[Range(0, 40)]	Allowed numeric range is 0 through 40.
[Column(TypeName = "decimal(3,2)")]	Use decimal precision/scale.
string?	The property may be null.
= new List<...>()	Start a collection as an empty list.
= DateTime.UtcNow	C# property initializer; object starts with UTC time.

Technician model concepts

```
[Key]
public int Id { get; set; }

[Required]
[StringLength(20)]
public string EmployeeCode { get; set; } = string.Empty;

[Range(0, 40)]
public int ExperienceYears { get; set; }

[Range(0, 5)]
[Column(TypeName = "decimal(3,2)")]
public decimal Rating { get; set; } = 5;

public string? Notes { get; set; }

public ICollection<ServiceRequest> ServiceRequests
    = new List<ServiceRequest>();
```

Important: [Required] is not the same thing as a C# default. For example, = DateTime.UtcNow initializes the C# object; it is not automatically a SQL DEFAULT constraint.

4. EF Core and Database Operations

EF Core is the bridge between C# and SQL Server. AppDbContext exposes DbSet properties that represent tables/query sources.

DbContext

```
public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options)
        : base(options) { }

    public DbSet<User> Users { get; set; }
    public DbSet<Technician> Technicians { get; set; }
    public DbSet<ServiceRequest> ServiceRequests { get; set; }
}
```

EF Core	SQL idea	Meaning
.Where(...)	WHERE	Filter rows.
.FirstOrDefaultAsync(...)	TOP 1 + WHERE	First match or null.
.AnyAsync(...)	EXISTS	Does a match exist?
.ToListAsync()	SELECT	Execute and return list.
.OrderBy(...)	ORDER BY	Sort.
.Include(...)	JOIN-like loading	Load related entity.
.Add(entity)	INSERT	Track new entity.
.Remove(entity)	DELETE	Track deletion.
property = value	UPDATE	Change tracked entity.
.SaveChangesAsync()	INSERT/UPDATE/DELETE	Persist tracked changes.

CRUD mental model

```
// READ
var techs = await _context.Technicians
    .Where(t => t.IsActive)
    .ToListAsync();

// INSERT
_context.Technicians.Add(technician);
await _context.SaveChangesAsync();

// UPDATE
technician.IsAvailable = false;
await _context.SaveChangesAsync();

// DELETE
_context.Technicians.Remove(technician);
await _context.SaveChangesAsync();
```

Migrations in Code First are typically created/applied with:

Migration commands

```
dotnet ef migrations add InitialCreate
dotnet ef database update
```

5. DTOs — Input and Output

DTO means Data Transfer Object. We use DTOs so the API does not need to expose the entire database entity and so the client only sends fields it is allowed to control.

Create DTO

```
public class CreateTechnicianDto
{
    [Required]
    public int UserId { get; set; }

    [Required]
    [StringLength(20)]
    public string EmployeeCode { get; set; } = string.Empty;

    [Required]
    [StringLength(100)]
    public string Specialization { get; set; } = string.Empty;

    [Range(0, 40)]
    public int ExperienceYears { get; set; }

    public string? CurrentLocation { get; set; }
    public string? Notes { get; set; }
}
```

Notice that the create DTO does not contain Rating, job counters, Status, CreatedAt, etc. Those are server-controlled values.

Entity → response DTO

```
return technicians.Select(t => new TechnicianDto
{
    Id = t.Id,
    UserId = t.UserId,
    FullName = t.User.FullName,
    Email = t.User.Email,
    PhoneNumber = t.User.PhoneNumber,
    EmployeeCode = t.EmployeeCode,
    Specialization = t.Specialization,
    ExperienceYears = t.ExperienceYears,
    Rating = t.Rating,
    TotalJobs = t.TotalJobs,
    ActiveJobs = t.ActiveJobs,
    CompletedJobs = t.CompletedJobs,
    IsAvailable = t.IsAvailable,
    IsActive = t.IsActive,
    Status = t.Status,
    CurrentLocation = t.CurrentLocation,
    JoiningDate = t.JoiningDate
}).ToList();
```

Mental model: Entity = database/storage shape. DTO = API communication shape.

6. Service Layer — Business Logic

The service is where we ask: 'Is this operation allowed according to our application's rules?' Controllers should stay thin.

Business rule example

```
public async Task<bool> CreateAsync(CreateTechnicianDto dto)
{
    var user = await _context.Users
        .FirstOrDefaultAsync(x => x.Id == dto.UserId);

    if (user == null)
        return false;

    if (user.Role != "Technician")
        return false;

    var codeExists = await _context.Technicians
        .AnyAsync(x => x.EmployeeCode == dto.EmployeeCode);

    if (codeExists)
        return false;

    // create entity and save...
}
```

- Find the selected User.
- If User does not exist → stop.
- If User is not a Technician → stop.
- If EmployeeCode already exists → stop.
- Otherwise create the Technician.

Work-status business rule

```
if (dto.Status != "Available" &&
    dto.Status != "Busy" &&
    dto.Status != "On Leave")
{
    return false;
}

technician.Status = dto.Status;
technician.IsAvailable = dto.Status == "Available";
technician.UpdatedAt = DateTime.UtcNow;
```

This is business logic because the system decides which status values are valid and how IsAvailable must follow Status.

7. Controller — Routing, Binding and Responses

The controller is the public HTTP entry point. ASP.NET Core chooses an action using the route and HTTP verb, binds JSON to the DTO, checks authorization, and then the action calls the service.

Controller pattern

```
[Authorize]
[ApiController]
[Route("api/[controller]")]
public class TechniciansController : ControllerBase
{
    private readonly TechnicianService _technicianService;

    public TechniciansController(TechnicianService technicianService)
    {
        _technicianService = technicianService;
    }

    [Authorize(Roles = "Owner,Manager")]
    [HttpGet]
    public async Task<IActionResult> GetAll()
    {
        var technicians =
            await _technicianService.GetAllAsync();

        return Ok(new
        {
            success = true,
            count = technicians.Count,
            data = technicians
        });
    }
}
```

Attribute	Resulting endpoint
[HttpGet]	GET /api/Technicians
[HttpGet("{id}")]	GET /api/Technicians/5
[HttpPost]	POST /api/Technicians
[HttpPut("{id}")]	PUT /api/Technicians/5
[HttpPatch("{id}/deactivate")]	PATCH /api/Technicians/5/deactivate

Common responses

```
if (!ModelState.IsValid)
    return BadRequest(ModelState);

return Ok(new { success = true, data = result });

return StatusCode(201, new {
    success = true,
    message = "Created successfully."
});

return NotFound(new {
    success = false,
    message = "Not found."
});
```

HTTP basics: 200 = success, 201 = created, 400 = bad input/business failure, 401 = unauthenticated, 403 = authenticated but forbidden, 404 = not found.

8. JWT Authentication and Authorization

Authentication: Who are you? Authorization: What can you do?

Login flow

```
POST /api/Auth/login

{
  "email": "owner@example.com",
  "password": "your-password"
}
```

```
Server:
1. Find user.
2. Verify password.
3. Create JWT.
4. Put identity/role claims in token.
5. Return token.

Later:
Authorization: Bearer <JWT>
```

Our controllers read the current user ID from the authenticated identity:

Current user ID

```
var userId = int.Parse(
    User.FindFirst(ClaimTypes.NameIdentifier)!.Value
);
```

Role authorization

```
[Authorize(Roles = "Owner,Manager")]
[HttpGet]
public async Task<IActionResult> GetAll()
{
    ...
}
```

Protection	Meaning
[Authorize]	User must be authenticated.
[Authorize(Roles="Customer")]	Authenticated user must have Customer role.
JWT NameIdentifier claim	Used to identify the current user.
CustomerId check in query	Protects ownership of customer records.

Important security rule: never trust a customerId sent by the browser as proof of ownership. Get the user ID from the JWT, then query the database using both request ID and current customer ID.

9. Include and ThenInclude — Deep Explanation

This is a core EF Core concept in our Service Request queries.

Request → Technician → User

```
var requests = await _context.ServiceRequests
    .Include(sr => sr.Technician)
    .ThenInclude(t => t!.User)
    .ToListAsync();
```

Read it step by step:

- Start with ServiceRequest.
- Include Technician: load the technician related to each request.
- ThenInclude User: from that Technician, continue to its User.
- Now the code can read sr.Technician.User.FullName.

Why two levels are needed

```
ServiceRequest
      |
      | TechnicianId
      v
Technician
      |
      | UserId
      v
User
```

Why can we sometimes write CustomerName = sr.Customer.FullName without ThenInclude? Because in the entity model used by that query, sr.Customer is already the User/customer navigation object. There is no second object such as CustomerProfile.User that must be traversed. The exact Include requirement depends on the navigation properties in the actual models.

Include loads related data; it does not create the relationship. The relationship comes from FK/navigation properties.

10. Technician Module — Complete Flow

Create technician example

```
POST /api/Technicians

{
  "userId": 12,
  "employeeCode": "TECH-001",
  "specialization": "AC Repair",
  "experienceYears": 4,
  "currentLocation": "Rajkot",
  "notes": "Experienced technician"
}
```

Create service logic:

Server-controlled defaults

```
var user = await _context.Users
    .FirstOrDefaultAsync(x => x.Id == dto.UserId);

if (user == null)
    return false;

if (user.Role != "Technician")
    return false;

var codeExists = await _context.Technicians
    .AnyAsync(x => x.EmployeeCode == dto.EmployeeCode);

if (codeExists)
    return false;

var technician = new Technician
{
    UserId = dto.UserId,
    EmployeeCode = dto.EmployeeCode,
    Specialization = dto.Specialization,
    ExperienceYears = dto.ExperienceYears,
    CurrentLocation = dto.CurrentLocation,
    Notes = dto.Notes,
    Status = "Available",
    IsAvailable = true,
    IsActive = true,
    CreatedAt = DateTime.UtcNow,
    JoiningDate = DateTime.UtcNow
};

_context.Technicians.Add(technician);
await _context.SaveChangesAsync();
```

Dropdown uses only active and available technicians so managers do not accidentally assign someone inactive/unavailable.

Dropdown query

```
var technicians = await _context.Technicians
    .Include(t => t.User)
    .Where(t => t.IsActive && t.IsAvailable)
    .OrderBy(t => t.User.FirstName)
    .ToListAsync();
```

11. Service Request Module — Complete Flow

A service request is the central object in the field-service workflow. It starts with a customer and service, then gets assigned to a technician, completed, reported, invoiced and eventually closed.

Create request

```
public async Task<bool> CreateAsync(
    int customerId,
    CreateServiceRequestDto dto)
{
    var service = await _context.Services
        .FirstOrDefaultAsync(s =>
```

```
            s.Id == dto.ServiceId &&
            s.IsActive);

    if (service == null)
        return false;

    var request = new ServiceRequest
    {
        CustomerId = customerId,
        ServiceId = dto.ServiceId,
        ProblemDescription = dto.ProblemDescription,
        ServiceAddress = dto.ServiceAddress,
        PreferredDate = dto.PreferredDate,
        Notes = dto.Notes,
        Status = "Pending",
        EstimatedPrice = 0,
        CreatedAt = DateTime.UtcNow
    };

    _context.ServiceRequests.Add(request);
    await _context.SaveChangesAsync();

    return true;
}
```

Why check IsActive? Because a customer should not create a request for a service management has disabled.

Ownership protection

```
var request = await _context.ServiceRequests
    .FirstOrDefaultAsync(sr =>
        sr.Id == requestId &&
        sr.CustomerId == customerId);
```

This combines two conditions: the request must have the requested ID AND belong to the current customer. That is much safer than querying only by requestId.

Cancel rule

```
if (request.Status != "Pending")
    return false;

request.Status = "Cancelled";
request.UpdatedAt = DateTime.UtcNow;

await _context.SaveChangesAsync();
```

12. Dashboards — How Output Is Produced

A dashboard usually calculates values from existing records instead of storing every number in a separate table.

Customer dashboard calculation

```
var requests = await _context.ServiceRequests
    .Where(x => x.CustomerId == customerId)
    .ToListAsync();

return new
{
    TotalRequests = requests.Count,
    PendingRequests = requests.Count(x =>
        x.Status == "Pending"),
    CompletedRequests = requests.Count(x =>
        x.Status == "Completed"),
    CancelledRequests = requests.Count(x =>
        x.Status == "Cancelled"),
    ActiveRequests = requests.Count(x =>
        x.Status != "Completed" &&
        x.Status != "Cancelled")
};
```

Example: 4 requests = 2 Pending + 1 Completed + 1 Cancelled. TotalRequests = 4 and ActiveRequests = 2.

13. Complete Business Workflow

Step	Actor	Action
1	Customer	Login and create service request

Step	Actor	Action
2	Manager	Review pending request
3	Manager	Assign technician
4	Technician	Accept job
5	Technician	Travel to customer
6	Technician	Start work
7	Technician	Submit work report
8	Manager	Review report
9	Manager	Approve report
10	Manager	Generate invoice
11	Customer	Pay invoice
12	Manager	Close request

This sequence is the project's defined business workflow. ■filecite■turn13file2■L359-L372■

Status	Meaning
Pending	Created by customer.
UnderReview	Management reviewing.
Assigned	Technician assigned.
Accepted	Technician accepted.
OnTheWay	Technician travelling.
InProgress	Technician working.
WorkCompleted	Technician submitted completion/report.
Completed	Management approved/completed work stage.
Invoiced	Invoice generated.
Paid	Customer paid.
Closed	Management closed request.
Cancelled	Request cancelled under allowed rules.

The status rules in the frontend reference define role-based transitions such as Pending → UnderReview for management, Assigned → Accepted for technicians, and Completed → Invoiced for management. ■filecite■turn11file8■L698-L754■

14. API URLs — Confirmed vs Target

Confirmed/discussed Technician routes:

Method	URL	Role	Purpose
GET	/api/Technicians	Owner, Manager	List technicians
GET	/api/Technicians/{id}	Owner, Manager	Get technician

Method	URL	Role	Purpose
POST	/api/Technicians	Owner	Create
PUT	/api/Technicians/{id}	Owner, Manager	Update
PATCH	/api/Technicians/{id}/deactivate	Owner	Deactivate
PATCH	/api/Technicians/{id}/toggle	Owner	Toggle active
PATCH	/api/Technicians/{id}/work-status	Owner, Manager	Change work status
GET	/api/Technicians/dropdown	Owner, Manager	Available technician dropdown
PATCH	/api/Technicians/my-status	Technician	Technician changes own status

Confirmed/discussed Service Request routes:

Method	URL	Role	Purpose
GET	/api/ServiceRequests	Owner, Manager	Management list
GET	/api/ServiceRequests/{id}	Role-specific	Get request by ID; role-specific action must be uniquely routed
GET	/api/ServiceRequests/my	Customer	My requests
POST	/api/ServiceRequests	Customer	Create request
PUT	/api/ServiceRequests/{id}	Customer	Update own pending request
PATCH	/api/ServiceRequests/{id}/cancel	Customer	Cancel own pending request
GET	/api/ServiceRequests/dashboard	Customer	Customer dashboard
GET	/api/ServiceRequests/management-dashboard	Owner, Manager	Management dashboard

Your Scalar test successfully returned HTTP 200 from /api/ServiceRequests/management-dashboard and showed TotalRequests, PendingRequests, AssignedRequests, InProgressRequests, CompletedRequests and CancelledRequests.

Target/recommended API catalogue from the project API document

Module	Target endpoints
Authentication	POST /api/auth/login; POST /api/auth/register; POST /api/auth/logout; GET /api/auth/profile; PUT /api/auth/change-password
Users	GET/POST/PUT/DELETE /api/users; status; profile-image
Customer Addresses	GET/POST/PUT/DELETE /api/customer-addresses; /default
Services	GET/POST/PUT/DELETE /api/services; /status
Technicians	GET/POST/PUT; availability; jobs; dashboard
Service Requests	GET/POST/PUT; review; assign-technician; accept; start-travel; start-work; complete; close
Work Reports	GET/POST/PUT; approve; request-correction
Invoices	GET/POST/PUT; send; pay; customer invoices
Notifications	GET; GET by id; read; delete
Dashboard	customer; technician; manager; owner
Settings	GET; PUT

Module	Target endpoints
Status History	GET all; GET request timeline

The project API document labels this as a recommended list and states a total of 69 recommended APIs. ■filecite■turn13file1■L185-L216■ ■filecite■turn13file1■L218-L251■ ■filecite■turn13file1■L253-L286■

15. Scalar Testing — Exact Thinking

For every endpoint, ask five questions: What method? What URL? What role? What body/path/query? What response/status code?

GET example

```
GET
http://localhost:<your-port>/api/Technicians

Authorization:
Bearer <OWNER_OR_MANAGER_JWT>
```

POST example

```
POST
http://localhost:<your-port>/api/Technicians

Authorization:
Bearer <OWNER_JWT>

Body:
{
  "userId": 12,
  "employeeCode": "TECH-001",
  "specialization": "AC Repair",
  "experienceYears": 4,
  "currentLocation": "Rajkot",
  "notes": "Experienced technician"
}
```

PATCH example

```
PATCH
http://localhost:<your-port>/api/ServiceRequests/7/cancel

Authorization:
Bearer <CUSTOMER_JWT>
```

Status	Meaning
200	Successful operation
201	Created
400	Invalid input or business rule failed
401	Authentication missing/invalid
403	Authenticated but role forbidden
404	Resource not found
500	Unexpected server error

16. SQL Server Cheat Sheet

SQL	EF Core
SELECT * FROM Technicians	_context.Technicians.ToListAsync()
WHERE IsActive = 1	.Where(t => t.IsActive)
TOP 1 + WHERE	.FirstOrDefaultAsync(...)

SQL	EF Core
EXISTS	.AnyAsync(...)
ORDER BY	.OrderBy(...)
INSERT	Add(entity) + SaveChangesAsync()
UPDATE	Change properties + SaveChangesAsync()
DELETE	Remove(entity) + SaveChangesAsync()
JOIN-like related loading	Include(...)

Useful database debugging

```
SELECT * FROM Users;

SELECT * FROM Technicians
WHERE IsActive = 1;

SELECT * FROM ServiceRequests
WHERE CustomerId = 12;

SELECT Status, COUNT(*) AS Total
FROM ServiceRequests
GROUP BY Status;
```

GROUP BY: rows with the same value are placed into groups; COUNT/SUM/AVG/etc. then calculate a value per group. This is useful for dashboard statistics.

17. Validation vs Business Logic vs Authorization

Layer	Question	Example
Validation	Is the input shaped correctly?	[Required], [Range], [StringLength]
Business Logic	Is the operation allowed by business rules?	Only Pending requests can be cancelled.
Authorization	Is this user allowed to call the endpoint?	[Authorize(Roles="Customer")]

Controller protection

```
[Authorize(Roles = "Customer")]
[HttpPatch("{id:int}/cancel")]
public async Task<IActionResult> Cancel(int id)
{
    var customerId = int.Parse(
        User.FindFirst(ClaimTypes.NameIdentifier)!.Value);

    var result = await _service.CancelAsync(id, customerId);
    ...
}
```

Service business rule

```
if (request == null)
    return false;

if (request.Status != "Pending")
    return false;

request.Status = "Cancelled";
```

18. What Is Done and What Remains

Area	Position
Authentication / JWT / role authorization	Built and used by protected endpoints
Users foundation	Built

Area	Position
Customer Addresses	Built
Services	Built
Technicians	Built, including status/dropdown/dashboard
Service Requests	Built, including customer and management flow; assignment/reassignment/job workflow discussed
Scalar testing	Used to verify backend endpoints

Before the final React integration, the planned workflow still needs the remaining backend pieces:

1	**Work Reports**	**Technician submits report; manager reviews/approves.**
2	Invoices	Generate, view, send/pay and connect to request.
3	Notifications	Workflow notifications.
4	Status History	Persistent request timeline.
5	Settings	User preferences.
6	Final status-flow cleanup	Make backend states/transitions match the intended lifecycle.
7	React integration	Connect all completed APIs role-by-role.

The original roadmap places Work Reports, Invoice, Notifications, Dashboard/Settings and then frontend integration, testing and deployment. ■filecite■turn13file2■L464-L478■

19. Interview Explanation — Say It Like This

Simple project explanation

```
A customer logs in and receives a JWT.
The customer creates a service request.
The controller receives the request and gets the customer ID
from the JWT.
The service validates the selected service and business rules.
EF Core creates the ServiceRequest entity and SaveChangesAsync()
persists it to SQL Server.

Management reviews the request and assigns an available technician.
The technician works on the job and later submits a work report.
Management approves the report, generates an invoice,
the customer pays, and management closes the request.
```

Why DTOs? They control what the client can send/receive.

Why service layer? It keeps business rules out of controllers.

Why EF Core? It lets C# entities/LINQ communicate with SQL Server.

Why JWT? It lets the API authenticate requests and identify the current user/role.

Why foreign keys? They connect related rows without duplicating complete records.

Why Include/ThenInclude? To load related entities required by a query.

20. Final One-Page Revision Map

Core mental model

```
MODEL
  ↓
DbContext
  ↓
Migration
  ↓
SQL Server
```

```
DTO
  ↓
Controller
  ↓
Service
  ↓
DbContext
  ↓
Database
  ↓
JSON response

AUTH
Login → JWT → Bearer Token → Claims

AUTHORIZATION
[Authorize] → logged in
[Authorize(Roles="Owner")] → correct role

DATABASE
PK = identifies a row
FK = connects tables
Navigation property = C# relationship
Include = load related entity
ThenInclude = continue relationship
Where = filter
Any = existence check
FirstOrDefault = first match or null
ToList = execute list query
Add = insert tracking
SaveChanges = persist changes

BUSINESS
Customer
  ↓
Request
  ↓
Manager Review
  ↓
Technician
  ↓
Work Report
  ↓
Manager Approval
  ↓
Invoice
  ↓
Payment
  ↓
Close
```

Next module: Work Reports. Build it in the same sequence: Model → DTO → DbContext/migration if required → Service → Controller → Scalar test → deep explanation → then Invoice.

Source basis: the project's workflow guide, database design, API catalogue, and code/work discussed in this conversation. The database and API design documents are used as the project specification; target/recommended APIs are explicitly separated from confirmed/discussed endpoints.